

Labb 5 Ethical Hacking

Samuel Melin

5.1 SANS - Cloud Scanning

5.1.3 Cloud Scanning

a) I wanted to calculate how many potential public ipv4 addresses that could be scanned in Oracle in the eu-amsterdam-1 region.

First I ran this command to get the documented infrastrucuter from Oracle:

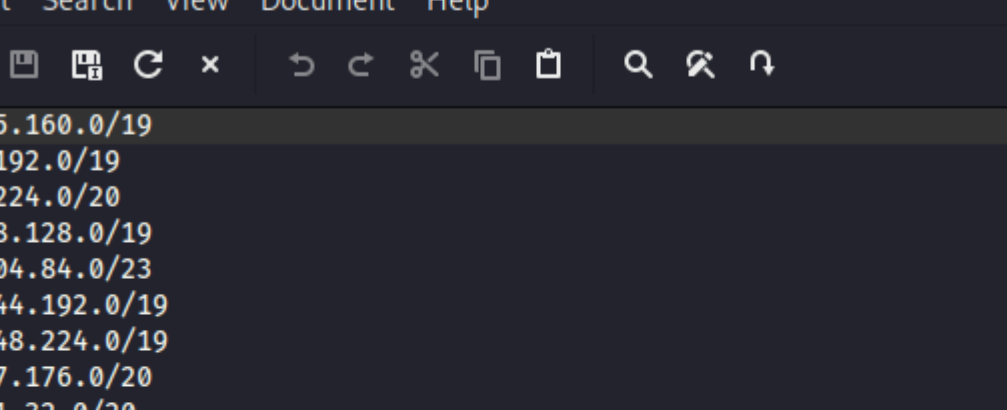
```
"wget https://docs.oracle.com/iaas/tools/public\_ip\_ranges.json  
(https://docs.oracle.com/iaas/tools/public\_ip\_ranges.json),"
```

Second I just check all the regions:

```
"jq < public_ip_ranges.json ':regions | .[] | .cidrs | .[].cidr' -r > oci.txt"
```

Third I checked for my specific region chosen:

```
"jq -r ':regions[] | select(.region == "eu-amsterdam-1") | .cidrs[].cidr'  
public_ip_ranges.json > oci1.txt"
```



~/.Desktop/Labb_5/oci1.txt - Mousepad

File Edit Search View Document Help

1 84.235.160.0/19
2 92.5.192.0/19
3 92.5.224.0/20
4 134.98.128.0/19
5 140.204.84.0/23
6 141.144.192.0/19
7 141.148.224.0/19
8 143.47.176.0/20
9 144.21.32.0/20
10 150.230.20.0/22
11 152.70.48.0/20
12 158.101.192.0/19
13 158.178.144.0/20
14 158.180.8.0/21
15 168.138.120.0/22
16 193.122.16.0/20
17 193.123.32.0/19
18 62.115.179.220/31
19 62.115.179.228/31
20 79.72.32.0/22
21 79.76.24.0/21
22 92.5.255.0/24
23 134.70.104.0/22

And the last step I used the python script "counttips.py" to calculate the IP:s:

```
kali@kali: ~/Desktop/Labb_5
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ cd Desktop/Labb_5
(kali@kali)-[~/Desktop/Labb_5]
$ python3 counttips.py
Usage: counttips.py <file with CIDR masks>
(kali@kali)-[~/Desktop/Labb_5]
$ python3 counttips.py oci1.txt
96904
```

b)

I ran masscan on 192.168.0.0/16 to test with rate at 1000 not overload the network. It took less than 5 minutes to get these results:

```
kali@kali: ~/Desktop/Labb_5
Session Actions Edit View Help

(kali㉿kali)-[~/Desktop/Labb_5]
$ sudo masscan 192.168.0.0/16 -p80,443,22 --rate 1000 --exclude 255.255.255.255
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2026-02-17 10:13:03 GMT
Initiating SYN Stealth Scan
Scanning 65536 hosts [3 ports/host]
Discovered open port 443/tcp on 192.168.0.1
Discovered open port 80/tcp on 192.168.0.1
65536 hosts scanned: 1 open, 65535 closed, 0 unreach, 0 state UNKNOWN, green default, green 1000
```

And here it discovered open ports on my router.

5.1.4 Cloud Bucket Discovery

a) No I can't find a tool that directly scans OCI buckets.

There for made a script:

```

kali@kali: ~/Desktop/Labb_5/5.1.4
Session Actions Edit View Help
GNU nano 8.7 bucketscan.py
import requests
import xml.etree.ElementTree as ET

def check_oci_bucket(namespace, region, bucket_name):
    url = f"https://{namespace}.compat.objectstorage.{region}.oraclecloud.com/{bucket_name}"
    r = requests.get(url)
    if r.status_code == 200:
        print(f"[+] {bucket_name} is public!")
        tree = ET.ElementTree(ET.fromstring(r.text))
        for elem in tree.getroot().iter():
            if elem.tag.endswith('Key'): print(f'File not found: {elem.text}')
    elif r.status_code == 403:
        print(f"[-] {bucket_name} exists but is private.")
    else:
        print(f"[!] {bucket_name} not found (404).")

check_oci_bucket(" ", " ", "PrivBucketFPB")

```

Result from private bucket:

```

kali@kali: ~/Desktop/Labb_5/5.1.4
Session Actions Edit View Help

(kali@kali)-[~/Desktop/Labb_5/5.1.4]
$ python3 bucketscan.py
[!] PrivBucketFPB not found (404).

```

Even tho this bucket exists in Oracle it still says it do not exist. This might be a security feture at Orcale to not give out a 403 repsonse.

Result from public bucket:

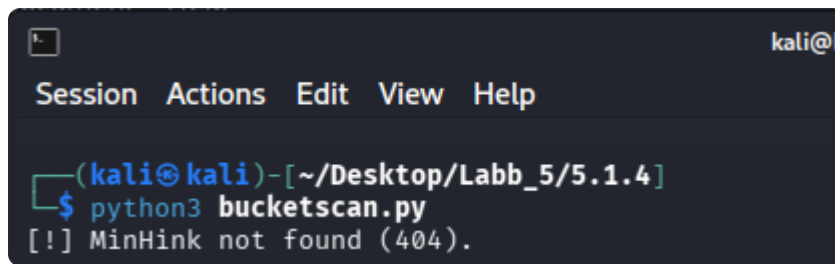
```

kali@kali: ~/Desktop/Labb_5/5.1.4
Session Actions Edit View Help

(kali@kali)-[~/Desktop/Labb_5/5.1.4]
$ python3 bucketscan.py
[+] PubBucketFPB is public!
File not found: Hello.txt

```

Result from non existing bucket:



```

kali@kali
Session Actions Edit View Help

(kali@kali)-[~/Desktop/Labb_5/5.1.4]
$ python3 bucketscan.py
[!] MinHink not found (404).

```

b) As stated before I couldnt not find the difference at this time. Im looking for a 403 repsonse when scanning for the PrivBucketFPB.

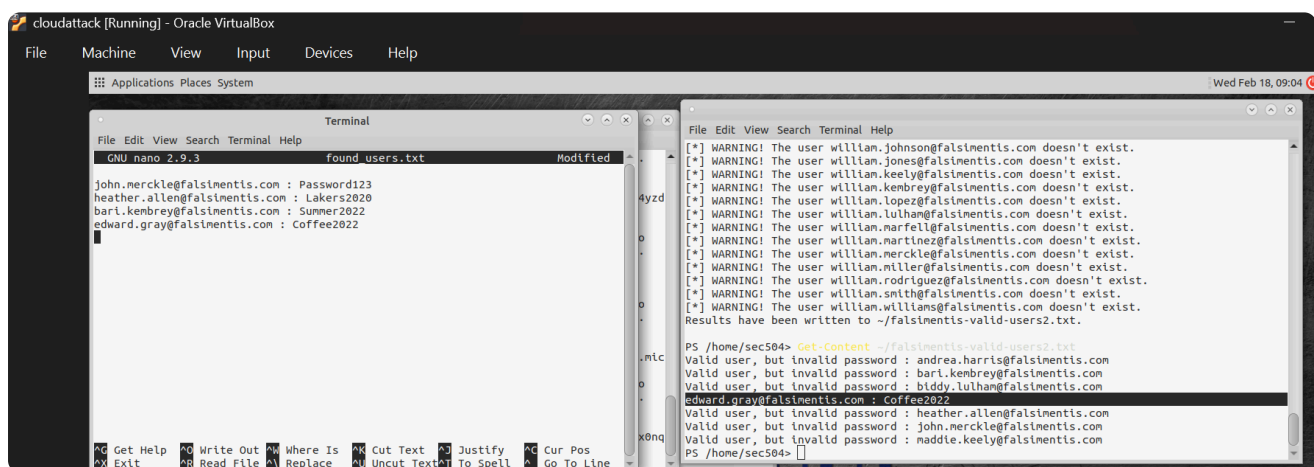
From my understanding when you get a 403 response you shall se that the bucket exists but you dont have access to it.

I tried with another private bucket also but it was the same result.

5.2 SANS Cloud Application Attacks

5.2.1 Microsoft 365 Password Attacks

a) After testing all 400 users I got the result:



```

cloudattack [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Applications Places System

Terminal
File Edit View Search Terminal Help
GNU nano 2.9.3 found_users.txt Modified
john.merckle@falsimentis.com : Password123
heather.allen@falsimentis.com : Lakers2020
bart.kembrey@falsimentis.com : Summer2022
edward.gray@falsimentis.com : Coffee2022

File Edit View Search Terminal Help
[*] WARNING! The user william.johnson@falsimentis.com doesn't exist.
[*] WARNING! The user william.jones@falsimentis.com doesn't exist.
[*] WARNING! The user william.keely@falsimentis.com doesn't exist.
[*] WARNING! The user william.kembrey@falsimentis.com doesn't exist.
[*] WARNING! The user william.lopez@falsimentis.com doesn't exist.
[*] WARNING! The user william.lulhan@falsimentis.com doesn't exist.
[*] WARNING! The user william.marfell@falsimentis.com doesn't exist.
[*] WARNING! The user william.martinez@falsimentis.com doesn't exist.
[*] WARNING! The user william.merckle@falsimentis.com doesn't exist.
[*] WARNING! The user william.miller@falsimentis.com doesn't exist.
[*] WARNING! The user william.rodriquez@falsimentis.com doesn't exist.
[*] WARNING! The user william.smith@falsimentis.com doesn't exist.
[*] WARNING! The user william.williams@falsimentis.com doesn't exist.
Results have been written to ~/falsimentis-valid-users2.txt.
PS /home/sec504> Get-Content ~/falsimentis-valid-users2.txt
Valid user, but invalid password : andrea.harris@falsimentis.com
Valid user, but invalid password : bari.kembrey@falsimentis.com
Valid user, but invalid password : bddy.lulhan@falsimentis.com
edward.gray@falsimentis.com : Coffee2022
Valid user, but invalid password : heather.allen@falsimentis.com
Valid user, but invalid password : john.merckle@falsimentis.com
Valid user, but invalid password : maddie.keely@falsimentis.com
PS /home/sec504>

```

b) If MFA is correctly setup with proper polices it is very hard to bypass. But what you can do as shown in the video "How to Find MFA Bypasses in Conditional Access Policies" is to do a MFASweep and check which service could be logged into without MFA and emulate the device in the browser and login to the account.

5.2.2 Microsoft 365 Password Attacks

a) According to the SANS workshop:

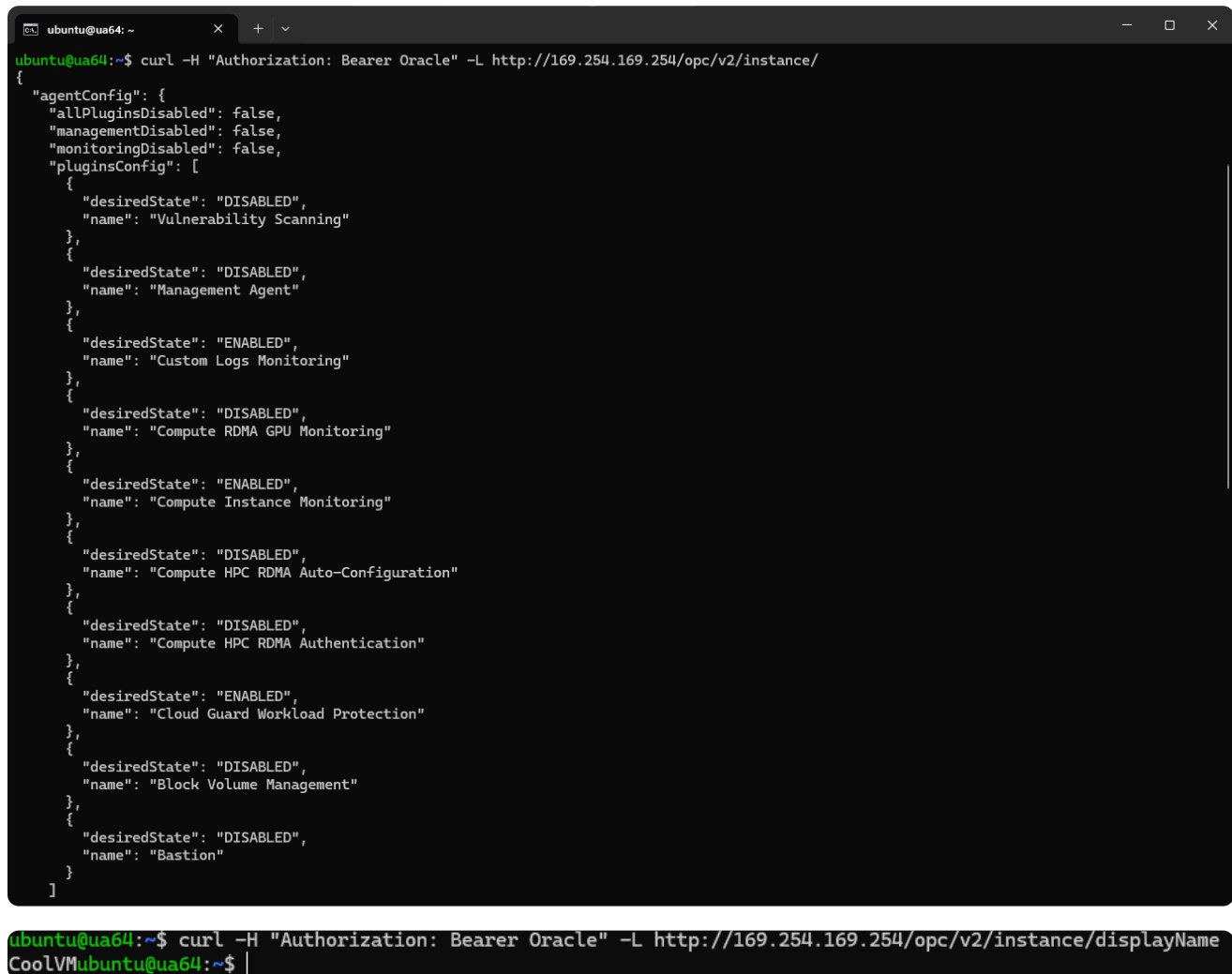
"The 169.254.169.254, IMDS (Instance Metadata Service) is a running instance for most clouds services.

The code that is running in a instance accesses IMDs to learn about its environment. It can disclose sensitive data such as authentication keys in some cloud provider(SANS workshop, 2023-01-25, SANS Institute)."

b) I used this command:

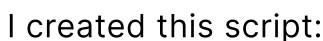
```
"curl -H "Authorization: Bearer Oracle" -L http://169.254.169.254/opc/v2/instance/
(http://169.254.169.254/opc/v2/instance/)."
```

I tried in my own cloud instance

A terminal window with a dark background. The prompt is 'ubuntu@ua64: ~'. The command entered is 'curl -H "Authorization: Bearer Oracle" -L http://169.254.169.254/opc/v2/instance/'. The output is a JSON object. The first part is 'agentConfig' with sub-objects for 'allPluginsDisabled', 'managementDisabled', 'monitoringDisabled', and 'pluginsConfig'. 'pluginsConfig' is an array of objects, each with 'desiredState' and 'name'. The names include 'Vulnerability Scanning', 'Management Agent', 'Custom Logs Monitoring', 'Compute RDMA GPU Monitoring', 'Compute Instance Monitoring', 'Compute HPC RDMA Auto-Configuration', 'Compute HPC RDMA Authentication', 'Cloud Guard Workload Protection', 'Block Volume Management', and 'Bastion'. The second command is 'curl -H "Authorization: Bearer Oracle" -L http://169.254.169.254/opc/v2/instance/displayName'. The output is 'CoolVMubuntu@ua64:~\$ |'.

```
ubuntu@ua64: ~  
ubuntu@ua64:~$ curl -H "Authorization: Bearer Oracle" -L http://169.254.169.254/opc/v2/instance/  
{  
  "agentConfig": {  
    "allPluginsDisabled": false,  
    "managementDisabled": false,  
    "monitoringDisabled": false,  
    "pluginsConfig": [  
      {  
        "desiredState": "DISABLED",  
        "name": "Vulnerability Scanning"  
      },  
      {  
        "desiredState": "DISABLED",  
        "name": "Management Agent"  
      },  
      {  
        "desiredState": "ENABLED",  
        "name": "Custom Logs Monitoring"  
      },  
      {  
        "desiredState": "DISABLED",  
        "name": "Compute RDMA GPU Monitoring"  
      },  
      {  
        "desiredState": "ENABLED",  
        "name": "Compute Instance Monitoring"  
      },  
      {  
        "desiredState": "DISABLED",  
        "name": "Compute HPC RDMA Auto-Configuration"  
      },  
      {  
        "desiredState": "DISABLED",  
        "name": "Compute HPC RDMA Authentication"  
      },  
      {  
        "desiredState": "ENABLED",  
        "name": "Cloud Guard Workload Protection"  
      },  
      {  
        "desiredState": "DISABLED",  
        "name": "Block Volume Management"  
      },  
      {  
        "desiredState": "DISABLED",  
        "name": "Bastion"  
      }  
    ]  
  }  
}  
  
ubuntu@ua64:~$ curl -H "Authorization: Bearer Oracle" -L http://169.254.169.254/opc/v2/instance/displayName  
CoolVMubuntu@ua64:~$ |
```

c) Yes it is, see image below:



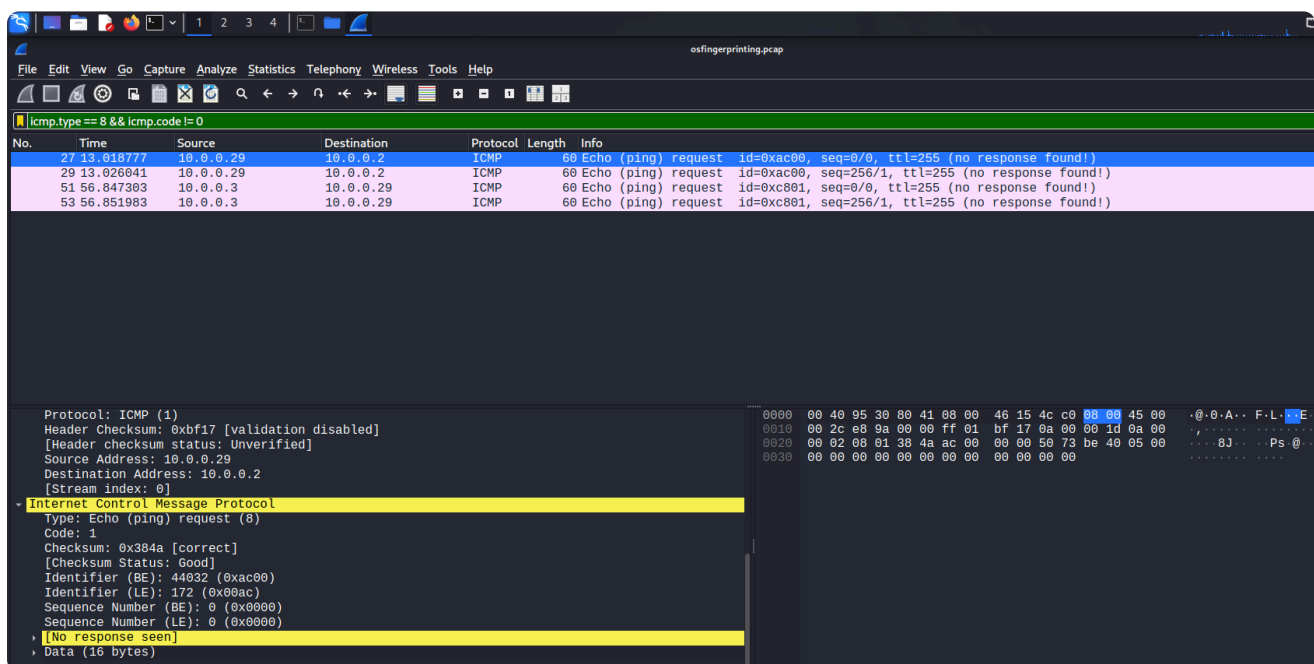
- **Strict Input Validation (Allow-listing):** The application should only allow requests to a predefined list of trusted domains. Requests to internal IPs (like 169.254.169.254 or 127.0.0.1) should be blocked.
- **Enforce IMDSv2:** By requiring a custom HTTP header or session token (as OCI does with "Bearer Oracle"), most simple SSRF attacks are stopped because the attacker often cannot control request headers, only the URL.
- Use local firewall rules (like iptables) to prevent the web server's service user from making any outbound connections to the metadata IP.

5.4 Wireshark - Attack and Reconnaissance Signatures

5.4.1 Task questions and help

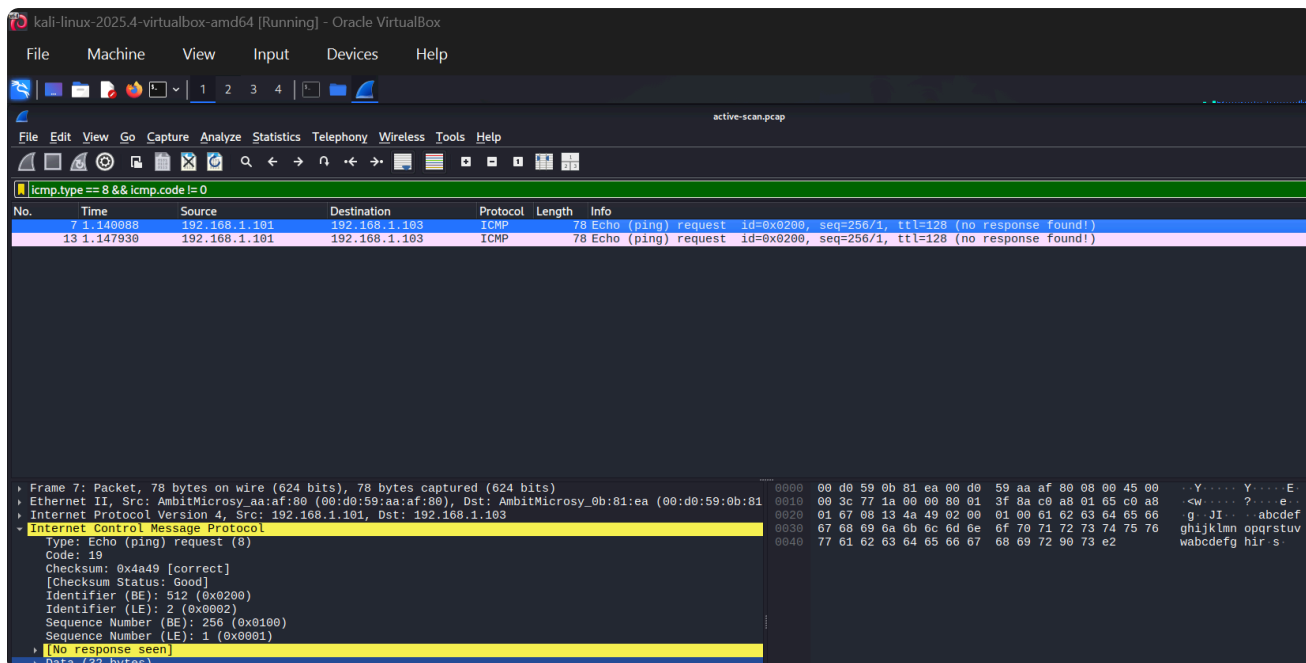
a)

Filter: icmp.type == 8 && icmp.code != 0

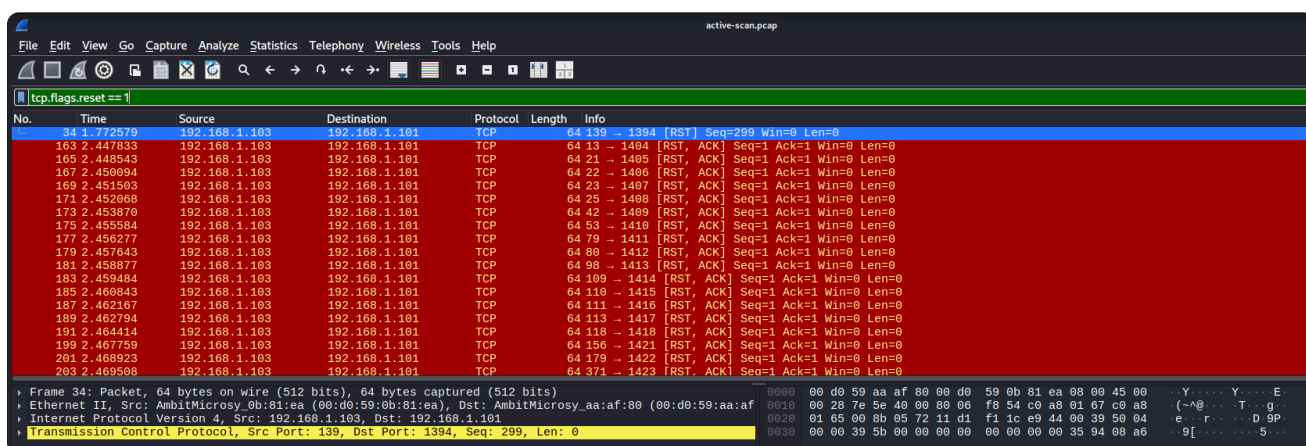


b)

Using the same filter i can see that the code is 19 which points to LanGuard scanner was used.



c)

Filter: `tcp.flags.reset == 1`

These means that there is an attempt to connect through the ports but they are closed or blocked by a firewall.

d)

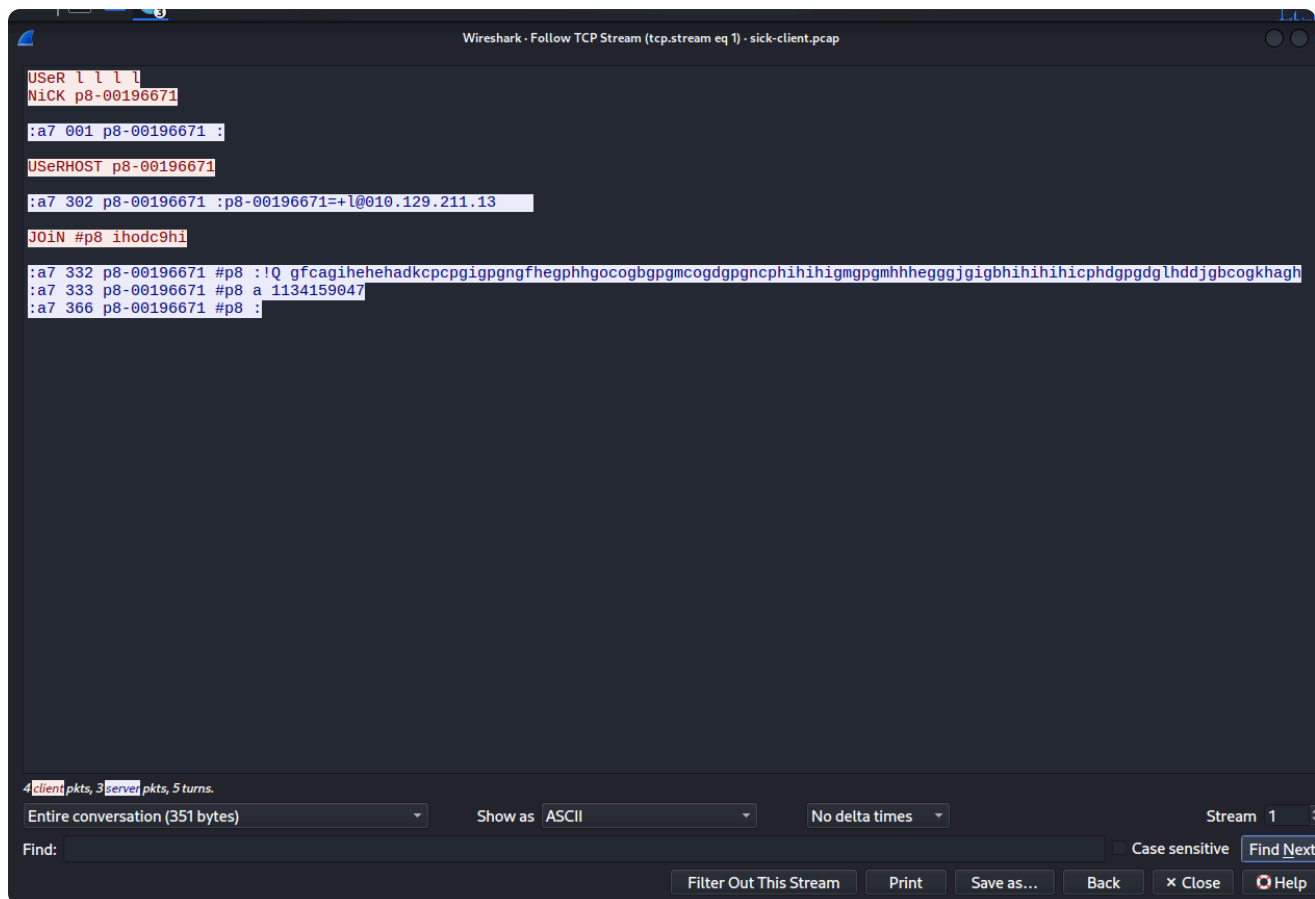
The scanner probed UDP ports and tried port 161 but was unreachable. This indicates that it is closed

The image shows a Wireshark packet capture analysis. The top toolbar and menu bar are visible. The packet list pane shows a single packet (No. 6) at time 0.574503, from source 192.168.1.103 to destination 192.168.1.101, protocol ICMP, length 74, with info 'Destination unreachable'. The packet details pane shows the following structure:

- Frame 6: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
- Ethernet II, Src: AmbitMicrosy_0b:81:ea (00:d0:59:0b:81:ea), Dst: AmbitMicrosy_aa:af:80 (00:d0:59:aa:af:80)
- Internet Protocol Version 4, Src: 192.168.1.103, Dst: 192.168.1.101
- Internet Control Message Protocol
 - Type: Destination unreachable (3)
 - Code: 3 (Port unreachable)
 - Checksum: 0xb6c2 [correct]
 - [Checksum Status: Good]
 - Unused: 00000000
 - Internet Protocol Version 4, Src: 192.168.1.101, Dst: 192.168.1.103
 - User Datagram Protocol, Src Port: 1393, Dst Port: 161
 - Source Port: 1393
 - Destination Port: 161
 - Length: 48
 - Checksum: 0x3ff8 [unverified]
 - [Checksum Status: Unverified]
 - [Stream index: 1]

e)

I used Statistics > Conversations > TCP and followed the stream with largest bytes and got this information:



Here I can see the IP and port its trying to connect to:

Ethernet . 1	IPv4 . 68	IPv6	TCP . 131	UDP . 1
Address A	Port A	Address B	Port B	Packets
10.129.211.13	1048	61.189.243.240	18067	12

- Nick: p8-00196671
- User: l
- Host/IP (bot's local IP): 10.129.211.13 (from USERHOST reply: +l@010.129.211.13 (mailto:+l@010.129.211.13))
- Botnet server IP:port: 61.189.243.240:18067 (from the TCP conversation you showed)

f) Here is the hierarchy:

Wireshark - Protocol Hierarchy Statistics - clientdying.pcap

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	514	100.0	105746	4,930	0	0	0	514
Ethernet	100.0	514	7.0	7430	346	0	0	0	514
Internet Protocol Version 4	100.0	514	9.7	10280	479	0	0	0	514
User Datagram Protocol	51.4	264	2.0	2112	98	0	0	0	264
Trivial File Transfer Protocol	50.2	258	1.0	1062	49	147	618	28	258
Data	21.6	111	53.7	56832	2,649	111	56832	2,649	111
Domain Name System	1.2	6	0.5	531	24	6	531	24	6
Transmission Control Protocol	48.6	250	5.1	5436	253	202	4476	208	250
Network News Transfer Protocol	0.2	1	0.1	68	3	1	68	3	1
NetBIOS Session Service	0.2	1	0.2	160	7	0	0	0	1
SMB (Server Message Block Protocol)	0.2	1	0.1	156	7	0	0	0	1
SMB Pipe Protocol	0.2	1	0.0	21	0	0	0	0	1
Distributed Computing Environment / Remote Procedure Call (DCE/RPC)	0.2	1	0.1	72	3	1	72	3	1
Internet Relay Chat	4.5	23	5.5	5767	268	23	5767	268	23
Hypertext Transfer Protocol	1.8	9	1.7	1800	83	9	1800	83	9
Distributed Computing Environment / Remote Procedure Call (DCE/RPC)	2.7	14	3.9	4112	191	8	528	24	14
ISystemActivator ISystemActivator Resolver	0.4	2	2.7	2880	134	2	2880	134	2
DCE/RPC Remote Management	0.8	4	0.5	560	26	4	560	26	4

1. Boot/startup happens

- The machine comes online and begins normal network initialization (typical early traffic is ARP/DHCP/DNS).

2. Malware is already on the machine

- Soon after startup, traffic appears that the host normally should not generate (per the lab hint): IRC, TFTP, RPC/SMB. That mismatch is the infection indicator.

3. IRC command-and-control connection

- The host opens a TCP connection to an IRC server.
-
- Meaning: the infected host is checking in to receive instructions.

4. TFTP file transfer begins (large volume)

- A lot of UDP traffic is TFTP (your Protocol Hierarchy shows ~50% as TFTP).
- TFTP is used to move files with minimal overhead.
- Meaning: the host is transferring payloads—most likely downloading additional malware or components.

5. SMB + DCE/RPC activity starts

- SMB/NetBIOS and DCE/RPC appear in the capture.
- On a client that “doesn’t talk RPC/SMB normally” (per hint), this suggests exploitation behavior.

- Meaning: the malware is attempting to scan, access, or exploit Windows services (often for spreading laterally).

6. Sustained malicious activity overloads the host

- Continuous IRC + heavy TFTP + RPC/SMB causes high packet processing and system work.
- Meaning: CPU load ramps up due to constant network-driven activity and malware execution.

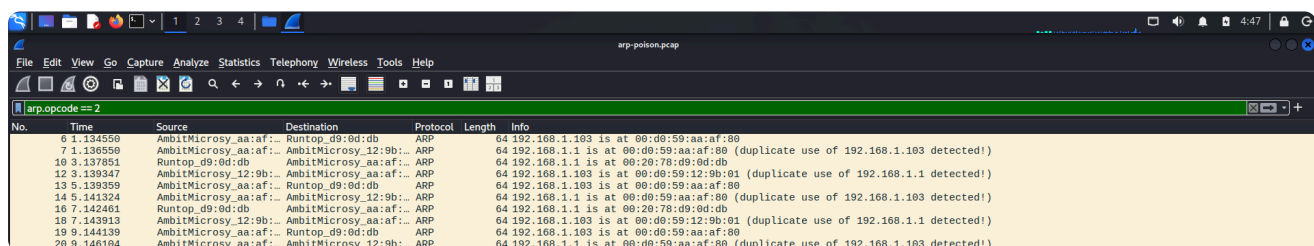
7. System becomes unusable

- After a few minutes, the workload is high enough that CPU hits 100% and the system locks up.
- In the pcap you don't see CPU directly; you infer it from the sudden, sustained abnormal traffic volume and then the capture ending / host becoming unresponsive.

g)

Using the filter "arp.opcode == 2"

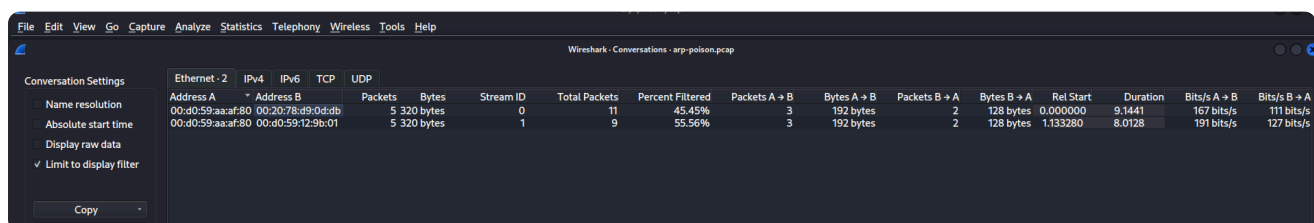
We can see duplicate use of 192.168.1.103 and 192.168.1.1:



The image shows a Wireshark packet capture window with the filter "arp.opcode == 2". The packet list shows several ARP requests. The first two packets (6 and 7) are from 192.168.1.103 to 192.168.1.1. The next two packets (10 and 12) are from 192.168.1.1 to 192.168.1.103. The following packets (13, 14, 16, 17, 19, 20) are from 192.168.1.103 to 192.168.1.1. The packet details pane shows the ARP request details for the first packet, including the source and destination MAC addresses and the IP address being requested.

No.	Time	Source	Destination	Protocol	Length	Info
6	1.134550	AmbitMicrosy_aa:af:...	Runtop_d9:0d:db	ARP	64	192.168.1.103 is at 00:0d:59:aa:af:80
7	1.136550	AmbitMicrosy_aa:af:...	AmbitMicrosy_12:9b:...	ARP	64	192.168.1.1 is at 00:0d:59:aa:af:80 (duplicate use of 192.168.1.103 detected!)
10	3.137851	Runtop_d9:0d:db	AmbitMicrosy_aa:af:...	ARP	64	192.168.1.1 is at 00:20:78:d9:0d:db
12	3.139347	AmbitMicrosy_12:9b:...	AmbitMicrosy_aa:af:...	ARP	64	192.168.1.103 is at 00:0d:59:12:9b:01 (duplicate use of 192.168.1.1 detected!)
13	5.139359	AmbitMicrosy_aa:af:...	Runtop_d9:0d:db	ARP	64	192.168.1.103 is at 00:0d:59:aa:af:80
14	5.141324	AmbitMicrosy_aa:af:...	AmbitMicrosy_12:9b:...	ARP	64	192.168.1.1 is at 00:0d:59:aa:af:80 (duplicate use of 192.168.1.103 detected!)
16	7.142461	Runtop_d9:0d:db	AmbitMicrosy_aa:af:...	ARP	64	192.168.1.1 is at 00:20:78:d9:0d:db
17	7.143913	AmbitMicrosy_12:9b:...	AmbitMicrosy_aa:af:...	ARP	64	192.168.1.103 is at 00:0d:59:12:9b:01 (duplicate use of 192.168.1.1 detected!)
19	9.144139	AmbitMicrosy_aa:af:...	Runtop_d9:0d:db	ARP	64	192.168.1.103 is at 00:0d:59:aa:af:80
20	9.146184	AmbitMicrosy_aa:af:...	AmbitMicrosy_12:9b:...	ARP	64	192.168.1.1 is at 00:0d:59:aa:af:80 (duplicate use of 192.168.1.103 detected!)

Also I can see in this view that one MAC-address is talking to both sides:



The image shows the Wireshark Conversation window for the "arp-poisn.pcap" file. It displays the traffic between three MAC addresses: 00:0d:59:aa:af:80 (Address A), 00:20:78:d9:0d:db (Address B), and 00:0d:59:12:9b:01 (Address C). The table shows the number of packets and bytes exchanged between each pair of addresses.

Conversation Settings	Ethernet 2	IPv4	IPv6	TCP	UDP
Name resolution	Address A	Address B			
Absolute start time	00:0d:59:aa:af:80	00:20:78:d9:0d:db			
Display raw data	00:0d:59:aa:af:80	00:20:78:d9:0d:db			
Limit to display filter					

Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
0	11	45.45%	3	192 bytes	2	128 bytes	0.000000	9.1441	167 bits/s	111 bits/s
1	9	55.56%	3	192 bytes	2	128 bytes	1.133280	8.0128	191 bits/s	127 bits/s

So the attacker claims to be both 192.168.1.103 and 192.168.1.1 with the MAC 00:0d:59:aa:af:80.

The victims are 192.168.1.1 with MAC 00:20:78:d9:0d:db and 192.168.1.103 with MAC 00:0d:59:12:9b:01.

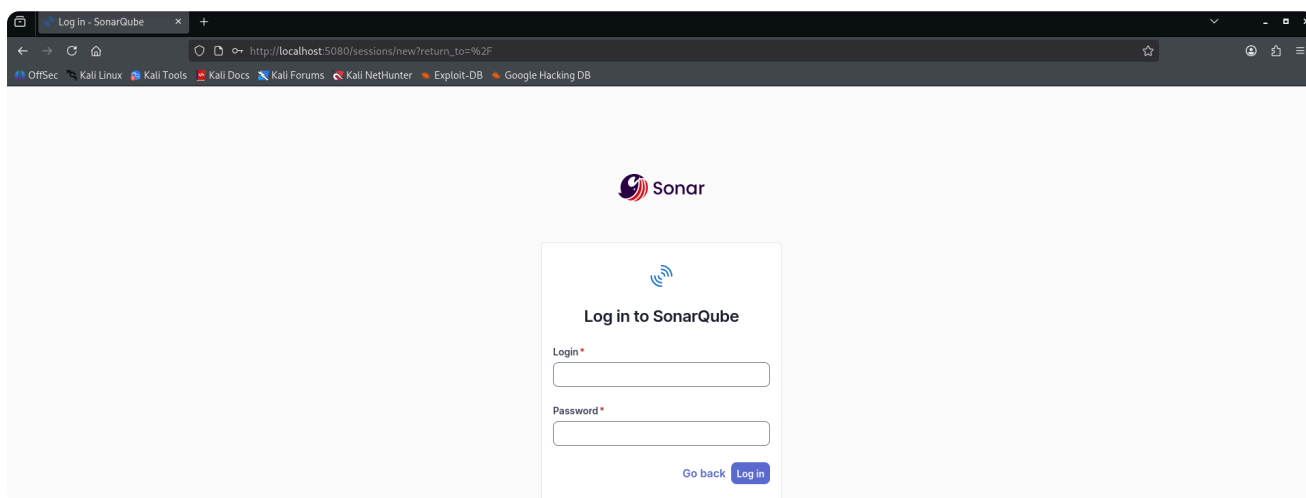
5.5 SANS - SEC565 Red Team Operations and Adversary Emulation

5.5.1 Forward SSH tunnel

I used the command

```
"ssh -L 5080:sqube-student.du.se:9000 h23samum@maggie.du.se  
(mailto:h23samum@maggie.du.se). -N  
"
```

Then i put in <http://localhost:5080> (<http://localhost:5080>) to access sqube-student.du.se:9000 (<http://sqube-student.du.se:9000>) in the browser:



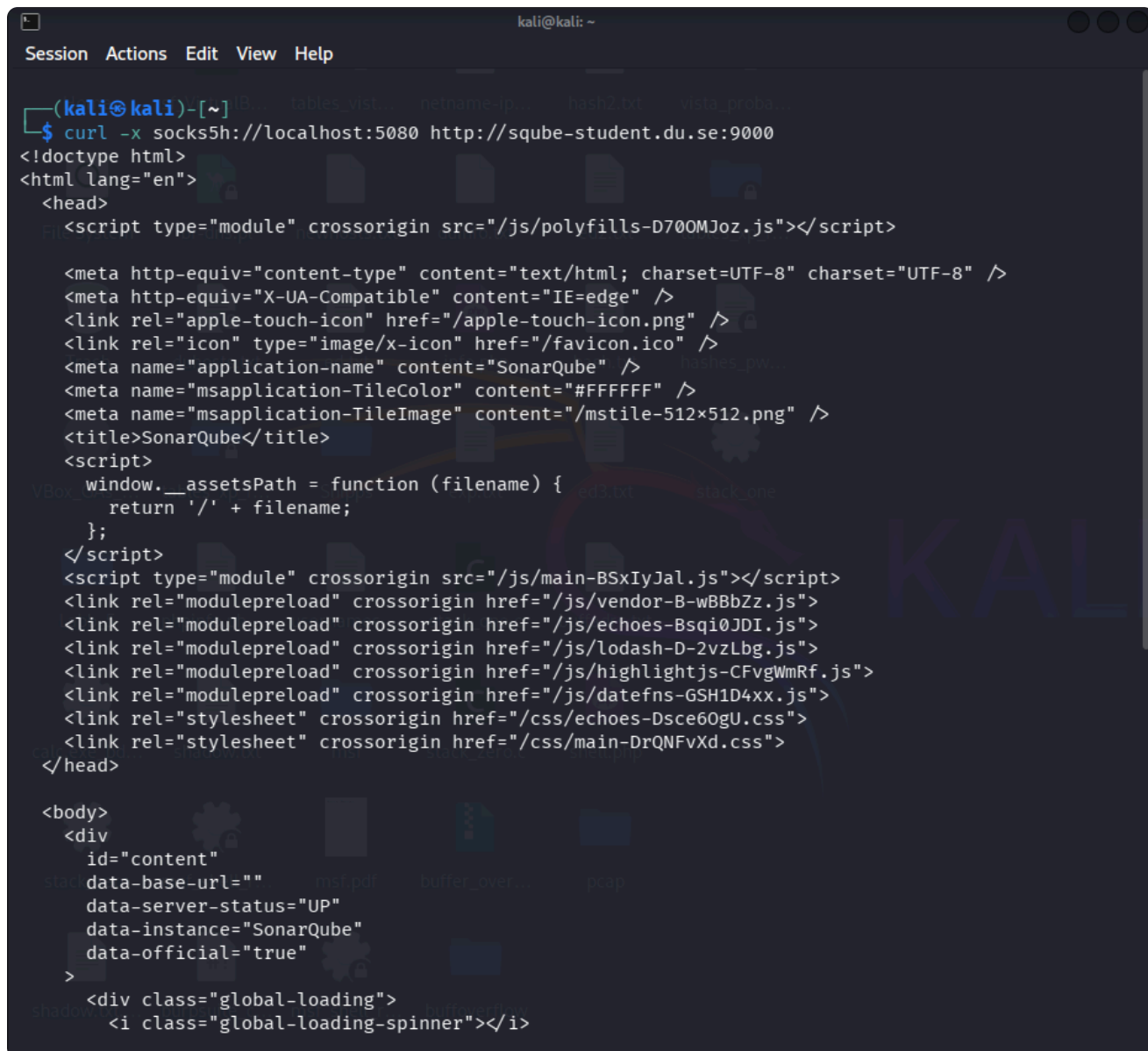
5.5.2 Socks proxy SSH tunnel

First i used the command

```
"ssh -D 5080 h23samum@maggie.du.se (mailto:h23samum@maggie.du.se). -N"
```

 to create a socks proxy tunnel.

I then used the command " `curl -x socks5h://localhost:5080 http://sqube-student.du.se:9000` (<http://sqube-student.du.se:9000>):



```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ curl -x socks5h://localhost:5080 http://sqube-student.du.se:9000
<!doctype html>
<html lang="en">
  <head>
    <script type="module" crossorigin src="/js/polyfills-D700MJoz.js"></script>

    <meta http-equiv="content-type" content="text/html; charset=UTF-8" charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <link rel="apple-touch-icon" href="/apple-touch-icon.png" />
    <link rel="icon" type="image/x-icon" href="/favicon.ico" />
    <meta name="application-name" content="SonarQube" />
    <meta name="msapplication-TileColor" content="#FFFFFF" />
    <meta name="msapplication-TileImage" content="/mstile-512x512.png" />
    <title>SonarQube</title>
  </head>
  <body>
    <div id="content" data-base-url="" data-server-status="UP" data-instance="SonarQube" data-official="true">
      <div class="global-loading">
        <i class="global-loading-spinner"></i>
      </div>
    </div>
  </body>
</html>

```

5.5.3 Double pivot SSH tunnel

Here I needed some help from Ilm to assist me with the command since I couldn't access my OCI instance with my key when I tried a less specified command.

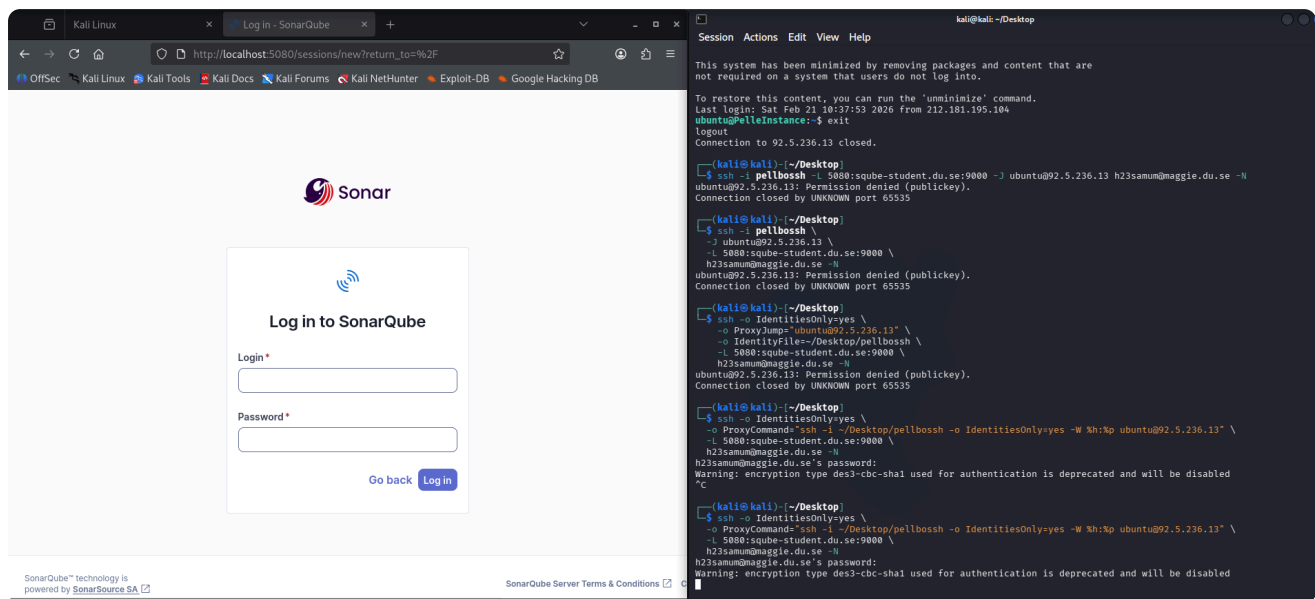
So the command became this:

```

ssh -o IdentitiesOnly=yes
-o ProxyCommand="ssh -i ~/Desktop/pellbossh -o IdentitiesOnly=yes -W %h:%p
ubuntu@92.5.236.13 (mailto:ubuntu@92.5.236.13)."
-L 5080:sqube-student.du.se:9000
h23samum@maggie.du.se (mailto:h23samum@maggie.du.se). -N

```

Warning: encryption type des3-cbc-sha1 used for authentication is deprecated and will be disabled

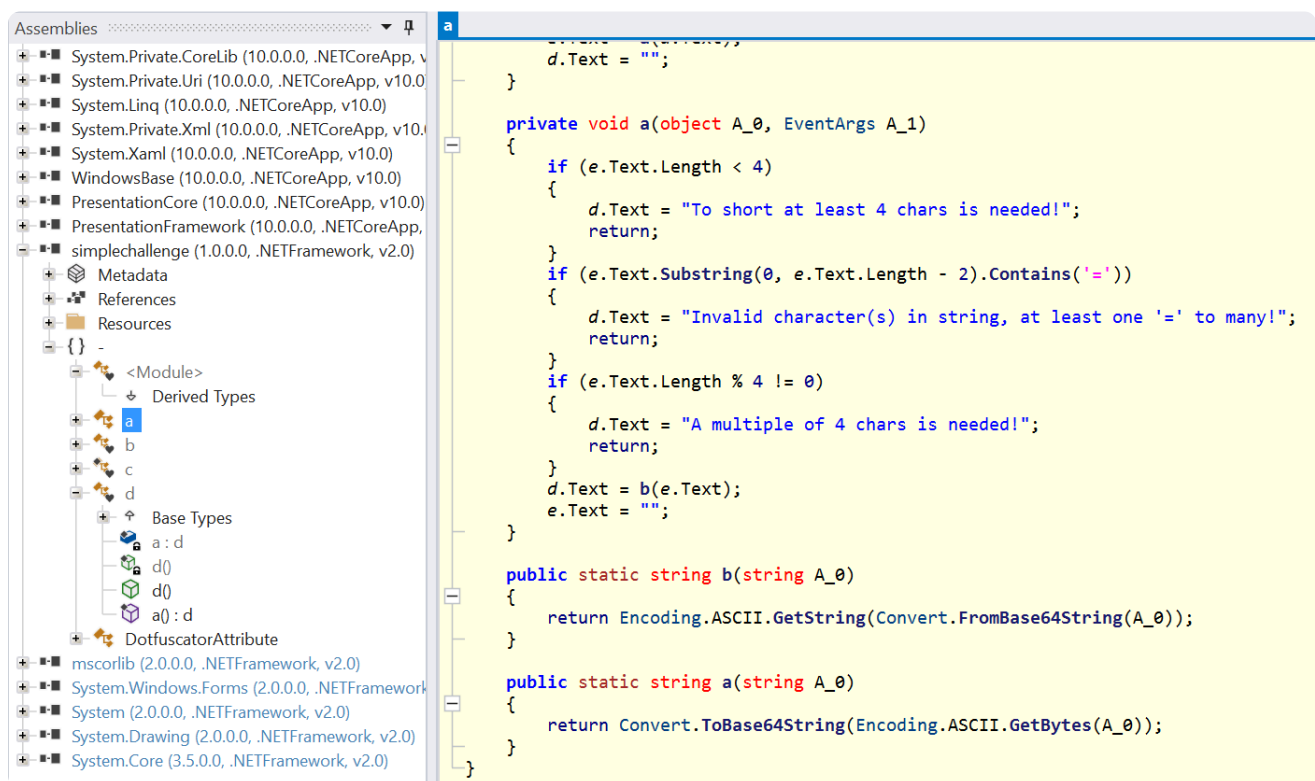


5.6 Reverse engineer managed code

At the first glance I could see that it used paddings with "=" which could indicate that the encryption is Base64.

With IISpy I could easily see how the encoding and decoding worked.

Encoding ASCII to Base64 and decoding from Base64 to ASCII:



5.7 Analysis of an unknown binary file

a)

When using DogBolt (<https://dogbolt.org/>) looking at Hex-Rays did some findings with the capabilities after unpacking the esh and looking up evilshell.c by Simpp on packetstorm:

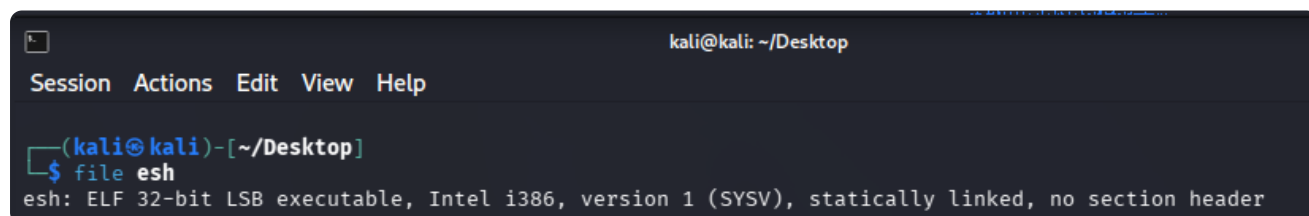
- Renames itself to look legitimate (/usr/sbin/lpinfo)
- Ignores termination signals (SIGINT, SIGTERM, etc.)
- Redirects file descriptors to /dev/null
- Runs in background (forks and parent exits)
- Disables shell history (HISTFILE=/dev/null)
- Opens a raw socket (ICMP)
- Listens for incoming packets
- Activates only when a specific packet is received:
- ICMP echo request (ping) that contains the value 1337.
- Creates a TCP socket
- Connects to a hardcoded IP address (IP_ADDRESS)
- Uses port 8000
- Redirects input/output to the socket (dup2)
- Launches /bin/bash

Uses uname() to collect:

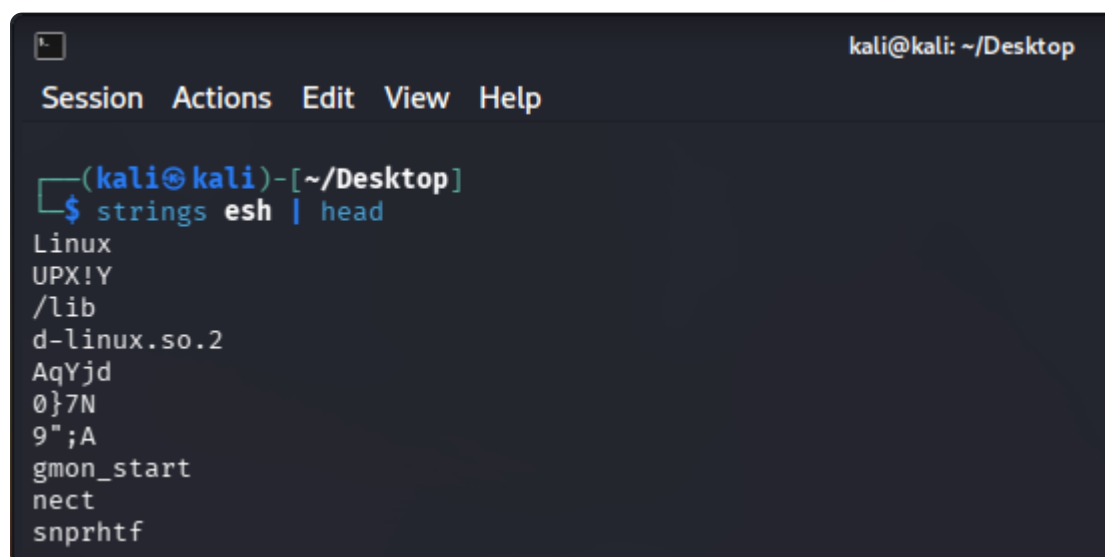
- kernel
- system name
- version
- hostname
- Sends a prompt (cmd->)
- Receives commands over the network
- Executes commands via the shell

b)

1. I use the command "file esh" and "strings esh | head" to examine what file type the esh file is :



```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ file esh
esh: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), statically linked, no section header
```



```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ strings esh | head
Linux
UPX!Y
/lib
d-linux.so.2
AqYjd
0}7N
9";A
gmon_start
nect
snprhtf
```

2. I confirm the file type with "upx -t esh":

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ upx -t esh

Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

testing esh [OK]

Tested 1 file.
```

3. I use "upx -d esh -o esh.unpacked"
" command to unpack it.

4. Then i verify the unpacked file:

```
kali@kali: ~/Desktop
Session Actions Edit View Help

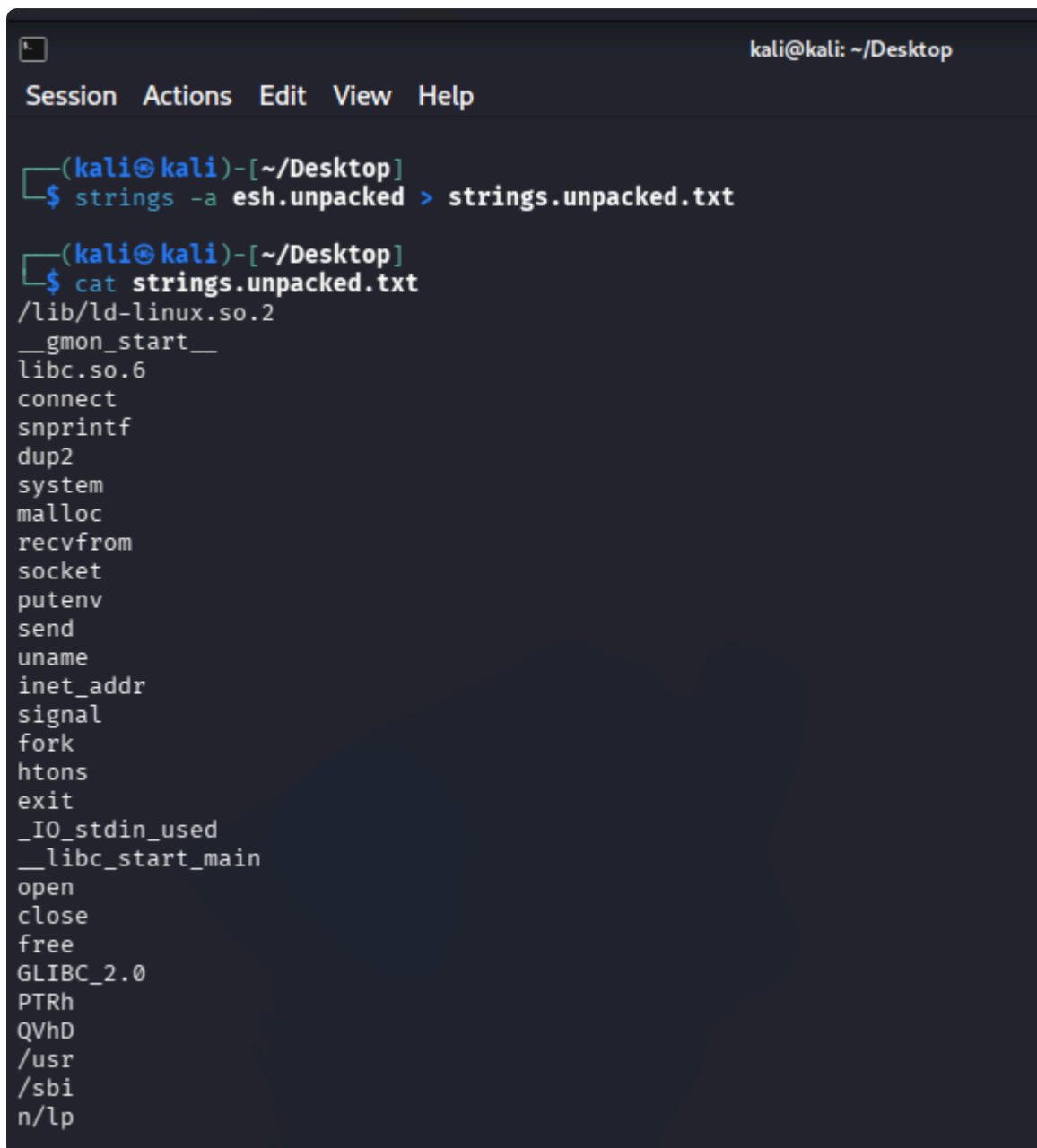
(kali@kali)-[~/Desktop]
$ file esh.unpacked
esh.unpacked: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.4.1, stripped
```

Also verifying it by using "readelf -h esh.unpacked | head -n 30"

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ readelf -h esh.unpacked | head -n 30
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                   ELF32
  Data:                     2's complement, little endian
  Version:                  1 (current)
  OS/ABI:                    UNIX - System V
  ABI Version:               0
  Type:                      EXEC (Executable file)
  Machine:                   Intel 80386
  Version:                   0x1
  Entry point address:       0x80486a0
  Start of program headers:  52 (bytes into file)
  Start of section headers: 5428 (bytes into file)
  Flags:                      0x0
  Size of this header:       52 (bytes)
  Size of program headers:   32 (bytes)
  Number of program headers:  7
  Size of section headers:   40 (bytes)
  Number of section headers: 26
  Section header string table index: 25
```

5. I extract the indicators and capabilities with the command "strings -a esh.unpacked > strings.unpacked.txt":



```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ strings -a esh.unpacked > strings.unpacked.txt

(kali@kali)-[~/Desktop]
$ cat strings.unpacked.txt
/lib/ld-linux.so.2
__gmon_start__
libc.so.6
connect
snprintf
dup2
system
malloc
recvfrom
socket
putenv
send
uname
inet_addr
signal
fork
htons
exit
_IO_stdin_used
__libc_start_main
open
close
free
GLIBC_2.0
PTRh
QVhD
/usr
/sbi
n/lp
```

And use dog bolt

c)

Most of the evidence is posted in the "b)" section.

Here is the full output of the strings.unpacked.txt and where we can see the code from Simpp 3vilSh3ll.

- strings.unpacked.txt:

/lib/ld-linux.so.2

gmon_start

libc.so.6

connect

snprintf

dup2

system

malloc

recvfrom

socket

putenv

send

uname

inet_addr

signal

fork

htons

exit

IO_stdin_used

__libc_start_main

open

close

free

GLIBC_2.0

PTRh

QVhD

/usr

/sbi

n/lp

info

[^]

[^]

[^]

crach

/dev/null

HISTFILE=/dev/null

/bin/bash

IP_ADDRESS

- BY SIMPP BACKDOORED SYSTEM INFO -

Noyau :

%s

System's name :

%s

Version :

%s

Process :

%s

Name host:

%s

cmd->

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

GCC: (GNU) 4.1.2 20061115 (prerelease) (Debian 4.1.1-21)

.shstrtab

.interp

.note.ABI-tag

.hash

.dynsym

.dynstr

.gnu.version

.gnu.version_r

.rel.dyn

.rel.plt

.init

.text

.fini

.rodata

.eh_frame

.ctors

.dtors

.jcr

.dynamic

.got

.got.plt

.data

.bss

.comment

Hex Rays:

3vilsh3ll by Simpp:

<https://packetstorm.news/files/id/69560> (<https://packetstorm.news/files/id/69560>).

d)

Using the command "sha256sum esh" I tried to search for the hash

"713cd81420de6561d9450b4882690fb1ef3d24661d27e8e495b23d3c5bd9f8bf" on the internet but couldn't find any answer.

By searching "BY SIMPP BACKDOORED SYSTEM INFO" from the strings.unpacked.txt on google I arrived at that it's pointing to "evilshell.c" which we can see the code for here:

<https://packetstorm.news/files/id/69560> (<https://packetstorm.news/files/id/69560>).

So this originated from Simpp that wrote this 3vilsh3ll.

5.8 Lab feedback

a) Yes I think it was very relevant with this lab since the cloud is popular. Also the other parts were relevant to get understanding how to reverse engineer etc! Very good lab!

The SANS material was very easy and good to follow.

b) I had some issues as seen when running into 5.5.3 when using my ssh key to double pivot as can be seen in that task. It might just be a confusion by me to have an instance that had keys enabled when doing this?

Either I solved it in the end by extended command.

```
ssh -i your-rsa-key ...
```

